

# DOCUMENTO DI SPECIFICA TECNICA:

## SpesaSmart Optimizer

**Versione:** 1.5 — Specifica Definitiva (FULL) **Data:** 12 aprile 2026 **Stato:** APPROVATO — Documento pronto per consegna allo sviluppatore

---

### 1. SCOPO E CONTESTO

Sviluppare un'applicazione web responsive (React) che permetta di ottimizzare il costo della spesa domestica calcolando il **"Costo Reale"**. Il sistema integra il prezzo dei prodotti con i costi logistici (distanza, tipo di carburante, traffico e consumo del veicolo) e suggerisce il percorso ottimale (TSP) tra più supermercati nel raggio di 20km dal CAP dell'utente.

---

### 2. STACK TECNOLOGICO

- **Frontend:** React.js (Vite) + Tailwind CSS + Lucide React (Icône).
  - **Backend & Hosting:** Firebase (Auth, Firestore, Hosting, Cloud Functions).
  - **Mappe & Geolocalizzazione:** Google Maps Platform (Distance Matrix API, Directions API con Traffic Layer).
  - **Data Scraping:** Node.js + Puppeteer (eseguito su Firebase Cloud Functions).
- 

### 3. ARCHITETTURA DEL DATABASE (Cloud Firestore)

#### 3.1 Collezione: `users`

- `uid`: string (Primary Key)
- `cap_predefinito`: string
- `veicolo`: { `tipo`, `consumo_medio` (L/100km), `prezzo_carburante_rif` }
- `family_id`: string (Per sincronizzazione liste tra membri della stessa famiglia)

#### 3.2 Collezione: `supermarkets`

- `sid`: string
- `insegna`: string (es. "Coop", "Esselunga", "Lidl")

- **coordinate**: geopoint (Lat/Lng)
- **servizi**: [ "spesa\_online", "ritiro\_in\_negozio" ]

### 3.3 Collezione: **daily\_prices** (Cache dei prezzi)

- **pid**: string (Reference prodotto)
  - **sid**: string (Reference supermercato)
  - **price**: number
  - **is\_offer**: boolean
  - **last\_updated**: timestamp
- 

## 4. DIFFERENZIATORI COMPETITIVI & FUNZIONALITÀ AVANZATE

### 4.1 Smart Trip Optimizer (Traffico Real-Time)

A differenza di app come DoveConviene, SpesaSmart calcola l'impatto del traffico sul consumo di carburante. Se il percorso verso il supermercato più economico è congestionato, il sistema suggerisce l'opzione con il miglior rapporto tempo/prezzo.

### 4.2 Prezzo Target & Scorte Stock

L'utente può contrassegnare prodotti come "Stockable" (es. caffè, detersivi). L'app invia una notifica push quando il prezzo di questi prodotti scende sotto una soglia prestabilita in un negozio vicino.

### 4.3 Family Cloud Sync (Sincronizzazione Bring!-style)

Le liste della spesa sono condivise. Se un coniuge aggiunge un prodotto mentre l'altro è in viaggio, l'algoritmo ricalcola istantaneamente se conviene fermarsi in un ulteriore punto vendita o se il costo del carburante extra annulla il risparmio.

---

## 5. LOGICA DI CALCOLO (Algoritmi)

- **Formula Costo Reale**:  $\text{Somma\_Prezzi} + ((\text{Km\_Totali} * \text{Consumo\_Veicolo} * \text{Prezzo\_Carburante}) / 100)$
- **Routing TSP**: Integrazione dell'algoritmo del "Commesso Viaggiatore" per ordinare le tappe (A -> B -> C) minimizzando i km totali partendo dalla posizione dell'utente.

---

## 6. PIANO DI RILASCIO E VINCOLI (Netlify)

1. **Deployment:** Continuous Integration tramite GitHub -> Netlify.
2. **Sicurezza:** Environment Variables obbligatorie per le chiavi API (Google Maps, Firebase).
3. **Acceptance Criteria:**
  - L'app deve mostrare il risparmio netto (in Euro) includendo la benzina.
  - Sincronizzazione tra due dispositivi diversi in meno di 2 secondi.

---

## 7. CONSEGNE OBBLIGATORIE DELLO SVILUPPATORE

- Codice sorgente completo e commentato.
- Documentazione delle Cloud Functions per lo scraping.
- Configurazione delle Security Rules di Firebase per garantire la privacy degli utenti.